TECHNICAL DOCUMENTATION & ENGINEERING STANDARDS

# Django Authentication & Authorization Standards

Custom User Models, JWT Stateless Tokens, RBAC Permissions, and Session Security

**Module:** django.contrib.auth / SimpleJWT | **Version:** 3.0 | **Owner:** Security Engineering | **Source Reference:** Django Authentication Docs

## 1. Custom User Model Standard

Apex strictly mandates substituting Django's default `auth.User` with a custom model extending `AbstractBaseUser` and `PermissionsMixin` before running the initial migration in any new repository:

```
# core/models.py
from django.contrib.auth.models import AbstractBaseUser, PermissionsMixin, BaseUserManager
from django.db import models
from django.utils import timezone

class EnterpriseUserManager(BaseUserManager):
    def create_user(self, email, password=None, **extra_fields):
        if not email:
            raise ValueError("The Email domain address is mandatory.")
        email = self.normalize_email(email)
        user = self.model(email=email, **extra_fields)
        user.set_password(password)
        user.save(using=self._db)
        return user

class EnterpriseUser(AbstractBaseUser, PermissionsMixin):
    email = models.EmailField(unique=True, db_index=True)
    first_name = models.CharField(max_length=60)
    last_name = models.CharField(max_length=60)
    department = models.CharField(max_length=100)
    is_active = models.BooleanField(default=True)
    is_staff = models.BooleanField(default=False)
    date_joined = models.DateTimeField(default=timezone.now)

    objects = EnterpriseUserManager()
    USERNAME_FIELD = 'email'
    REQUIRED_FIELDS = ['first_name', 'last_name']
```

In `config/settings.py`, set: `AUTH_USER_MODEL = 'core.EnterpriseUser'`.

## 2. JSON Web Token (JWT) API Authentication

Stateless microservices authenticate API requests using signed JWT tokens via `djangorestframework-simplejwt`:

```
# config/settings.py
from datetime import timedelta

REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ),
    'DEFAULT_PERMISSION_CLASSES': (
        'rest_framework.permissions.IsAuthenticated',
    ),
}

SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=15),
    'REFRESH_TOKEN_LIFETIME': timedelta(days=7),
    'ROTATE_REFRESH_TOKENS': True,
    'BLACKLIST_AFTER_ROTATION': True,
    'ALGORITHM': 'HS256',
    'SIGNING_KEY': config('JWT_SECRET_KEY'),
    'AUTH_HEADER_TYPES': ('Bearer',),
}
```

## 3. Role-Based Access Control (RBAC) and Object Permissions

Endpoints enforce granular access control using custom DRF permission classes:

```
from rest_framework import permissions

class IsDepartmentOwnerOrAdmin(permissions.BasePermission):
    def has_object_permission(self, request, view, obj):
        if request.user.is_staff or request.user.is_superuser:
            return True
        return obj.department == request.user.department and request.method in permissions.SAFE_METHODS
```

## 4. Session and Password Hardening

- **Password Hasher:** Enforce Argon2 or PBKDF2 with SHA-256 (minimum 600,000 iterations).
- **Session Cookies:** `SESSION_COOKIE_SECURE = True`, `SESSION_COOKIE_HTTPONLY = True`, `SESSION_COOKIE_SAMESITE = 'Lax'`.
- **Session Timeout:** `SESSION_COOKIE_AGE = 28800` (8 hours max idle session expiration).